

```

    sphyo=factor*yo;
    sphjpo=factor*jpo-sphjo/(2.*x);
    sphypo=factor*ypo-sphyo/(2.*x);
    sphno = n;
}

Doub Bessel::sphbesj(const Int n, const Doub x) {
Simple interface returning  $j_n(x)$ .
    if (n != sphno || x != xo) sphbes(n,x);
    return sphjo;
}

Doub Bessel::sphbesy(const Int n, const Doub x) {
Simple interface returning  $y_n(x)$ .
    if (n != sphno || x != xo) sphbes(n,x);
    return sphyo;
}

```

CITED REFERENCES AND FURTHER READING:

- Barnett, A.R., Feng, D.H., Steed, J.W., and Goldfarb, L.J.B. 1974, "Coulomb Wave Functions for All Real η and ρ ," *Computer Physics Communications*, vol. 8, pp. 377–395.[1]
- Temme, N.M. 1976, "On the Numerical Evaluation of the Ordinary Bessel Function of the Second Kind," *Journal of Computational Physics*, vol. 21, pp. 343–350[2]; 1975, *op. cit.*, vol. 19, pp. 324–337.[3]
- Numerical Recipes Software 2007, "Bessel Function Implementations," *Numerical Recipes Web-note No. 8*, at <http://www.nr.com/webnotes?8> [4]
- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nr.com/aands>, Chapter 10.[5]
- Thompson, I.J., and Barnett, A.R. 1987, "Modified Bessel Functions $I_\nu(z)$ and $K_\nu(z)$ of Real Order and Complex Argument, to Selected Accuracy," *Computer Physics Communications*, vol. 47, pp. 245–257.[6]
- Barnett, A.R. 1981, "An Algorithm for Regular and Irregular Coulomb and Bessel functions of Real Order to Machine Accuracy," *Computer Physics Communications*, vol. 21, pp. 297–314.
- Thompson, I.J., and Barnett, A.R. 1986, "Coulomb and Bessel Functions of Complex Arguments and Order," *Journal of Computational Physics*, vol. 64, pp. 490–509.

6.7 Spherical Harmonics

Spherical harmonics occur in a large variety of physical problems, for example, whenever a wave equation, or Laplace's equation, is solved by separation of variables in spherical coordinates. The spherical harmonic $Y_{lm}(\theta, \phi)$, $-l \leq m \leq l$, is a function of the two coordinates θ, ϕ on the surface of a sphere.

The spherical harmonics are orthogonal for different l and m , and they are normalized so that their integrated square over the sphere is unity:

$$\int_0^{2\pi} d\phi \int_{-1}^1 d(\cos \theta) Y_{l'm'}^*(\theta, \phi) Y_{lm}(\theta, \phi) = \delta_{l'l} \delta_{m'm} \quad (6.7.1)$$

Here the asterisk denotes complex conjugation.

Mathematically, the spherical harmonics are related to *associated Legendre polynomials* by the equation

$$Y_{lm}(\theta, \phi) = \sqrt{\frac{2l+1}{4\pi} \frac{(l-m)!}{(l+m)!}} P_l^m(\cos \theta) e^{im\phi} \quad (6.7.2)$$

By using the relation

$$Y_{l,-m}(\theta, \phi) = (-1)^m Y_{lm}^*(\theta, \phi) \quad (6.7.3)$$

we can always relate a spherical harmonic to an associated Legendre polynomial with $m \geq 0$. With $x \equiv \cos \theta$, these are defined in terms of the ordinary Legendre polynomials (cf. §4.6 and §5.4) by

$$P_l^m(x) = (-1)^m (1-x^2)^{m/2} \frac{d^m}{dx^m} P_l(x) \quad (6.7.4)$$

Be careful: There are alternative normalizations for the associated Legendre polynomials and alternative sign conventions.

The first few associated Legendre polynomials, and their corresponding normalized spherical harmonics, are

$P_0^0(x) = 1$	$Y_{00} = \sqrt{\frac{1}{4\pi}}$	(6.7.5)
$P_1^1(x) = -(1-x^2)^{1/2}$	$Y_{11} = -\sqrt{\frac{3}{8\pi}} \sin \theta e^{i\phi}$	
$P_1^0(x) = x$	$Y_{10} = \sqrt{\frac{3}{4\pi}} \cos \theta$	
$P_2^2(x) = 3(1-x^2)$	$Y_{22} = \frac{1}{4} \sqrt{\frac{15}{2\pi}} \sin^2 \theta e^{2i\phi}$	
$P_2^1(x) = -3(1-x^2)^{1/2}x$	$Y_{21} = -\sqrt{\frac{15}{8\pi}} \sin \theta \cos \theta e^{i\phi}$	
$P_2^0(x) = \frac{1}{2}(3x^2 - 1)$	$Y_{20} = \sqrt{\frac{5}{4\pi}} \left(\frac{3}{2} \cos^2 \theta - \frac{1}{2} \right)$	

There are many bad ways to evaluate associated Legendre polynomials numerically. For example, there are explicit expressions, such as

$$P_l^m(x) = \frac{(-1)^m (l+m)!}{2^m m! (l-m)!} (1-x^2)^{m/2} \left[1 - \frac{(l-m)(m+l+1)}{1!(m+1)} \left(\frac{1-x}{2} \right) + \frac{(l-m)(l-m-1)(m+l+1)(m+l+2)}{2!(m+1)(m+2)} \left(\frac{1-x}{2} \right)^2 - \dots \right] \quad (6.7.6)$$

where the polynomial continues up through the term in $(1-x)^{l-m}$. (See [1] for this and related formulas.) This is not a satisfactory method because evaluation of the polynomial involves delicate cancellations between successive terms, which alternate in sign. For large l , the individual terms in the polynomial become very much larger than their sum, and all accuracy is lost.

In practice, (6.7.6) can be used only in single precision (32-bit) for l up to 6 or 8, and in double precision (64-bit) for l up to 15 or 18, depending on the precision required for the answer. A more robust computational procedure is therefore desirable, as follows.

The associated Legendre functions satisfy numerous recurrence relations, tabulated in [1,2]. These are recurrences on l alone, on m alone, and on both l and m simultaneously. Most of the recurrences involving m are unstable, and so are dangerous for numerical work. The following recurrence on l is, however, stable (compare 5.4.1):

$$(l-m)P_l^m = x(2l-1)P_{l-1}^m - (l+m-1)P_{l-2}^m \quad (6.7.7)$$

Even this recurrence is useful only for moderate l and m , since the P_l^m 's themselves grow rapidly with l and quickly overflow. The spherical harmonics by contrast remain bounded — after all, they are normalized to unity (eq. 6.7.1). It is exactly the square-root factor in equation (6.7.2) that balances the divergence. So the right function to use in the recurrence relation is the renormalized associated Legendre function,

$$\tilde{P}_l^m = \sqrt{\frac{2l+1}{4\pi} \frac{(l-m)!}{(l+m)!}} P_l^m \quad (6.7.8)$$

Then the recurrence relation (6.7.7) becomes

$$\tilde{P}_l^m = \sqrt{\frac{4l^2-1}{l^2-m^2}} \left[x \tilde{P}_{l-1}^m - \sqrt{\frac{(l-1)^2-m^2}{4(l-1)^2-1}} \tilde{P}_{l-2}^m \right] \quad (6.7.9)$$

We start the recurrence with the closed-form expression for the $l = m$ function,

$$\tilde{P}_m^m = (-1)^m \sqrt{\frac{2m+1}{4\pi(2m)!}} (2m-1)!! (1-x^2)^{m/2} \quad (6.7.10)$$

(The notation $n!!$ denotes the product of all *odd* integers less than or equal to n .)

Using (6.7.9) with $l = m+1$, and setting $\tilde{P}_{m-1}^m = 0$, we find

$$\tilde{P}_{m+1}^m = x\sqrt{2m+3} \tilde{P}_m^m \quad (6.7.11)$$

Equations (6.7.10) and (6.7.11) provide the two starting values required for (6.7.9) for general l .

The function that implements this is

`plegendre.h`

`Doub plegendre(const Int l, const Int m, const Doub x) {`
 Computes the renormalized associated Legendre polynomial $\tilde{P}_l^m(x)$, equation (6.7.8). Here m and l are integers satisfying $0 \leq m \leq l$, while x lies in the range $-1 \leq x \leq 1$.

```
static const Doub PI=3.141592653589793;
Int i,ll;
Doub fact,oldfact,p11,pmm,pmmp1,omx2;
if (m < 0 || m > 1 || abs(x) > 1.0)
    throw("Bad arguments in routine plgndr");
pmm=1.0;                                Compute  $\tilde{P}_m^m$ .
if (m > 0) {
    omx2=(1.0-x)*(1.0+x);
    fact=1.0;
    for (i=1;i<=m;i++) {
        pmm *= omx2*fact/(fact+1.0);
        fact += 2.0;
    }
}
pmm=sqrt((2*m+1)*pmm/(4.0*PI));
```

```

if (m & 1)
    pmm=-pmm;
if (l == m)
    return pmm;
else {
    Compute  $\tilde{P}_{m+1}^m$ .
    pmp1=x*sqrt(2.0*m+3.0)*pmm;
    if (l == (m+1))
        return pmp1;
    else {
        Compute  $\tilde{P}_l^m$ ,  $l > m + 1$ .
        oldfact=sqrt(2.0*m+3.0);
        for (ll=m+2; ll<=l; ll++) {
            fact=sqrt((4.0*ll*ll-1.0)/(ll*ll-m*m));
            pll=(x*pmp1-pmm/oldfact)*fact;
            oldfact=fact;
            pmm=pmp1;
            pmp1=pll;
        }
        return pll;
    }
}
}

```

Sometimes it is convenient to have the functions with the standard normalization, as defined by equation (6.7.4). Here is a routine that does this. Note that it will overflow for $m \gtrsim 80$, or even sooner if $l \gg m$.

`Doub plgndr(const Int l, const Int m, const Doub x)`

`plegendre.h`

Computes the associated Legendre polynomial $P_l^m(x)$, equation (6.7.4). Here m and l are integers satisfying $0 \leq m \leq l$, while x lies in the range $-1 \leq x \leq 1$. These functions will overflow for $m \gtrsim 80$.

```

{
    const Doub PI=3.141592653589793238;
    if (m < 0 || m > l || abs(x) > 1.0)
        throw("Bad arguments in routine plgndr");
    Doub prod=1.0;
    for (Int j=l-m+1; j<=l+m; j++)
        prod *= j;
    return sqrt(4.0*PI*prod/(2*l+1))*plegendre(l,m,x);
}

```

6.7.1 Fast Spherical Harmonic Transforms

Any smooth function on the surface of a sphere can be written as an expansion in spherical harmonics. Suppose the function can be well-approximated by truncating the expansion at $l = l_{\max}$:

$$\begin{aligned}
 f(\theta_i, \phi_j) &= \sum_{l=0}^{l_{\max}} \sum_{m=-l}^{m=l} a_{lm} Y_{lm}(\theta_i, \phi_j) \\
 &= \sum_{l=0}^{l_{\max}} \sum_{m=-l}^{m=l} a_{lm} \tilde{P}_l^m(\cos \theta_i) e^{im\phi_j}
 \end{aligned} \tag{6.7.12}$$

Here we have written the function evaluated at one of N_θ sample points θ_i and one of N_ϕ sample points ϕ_j . The total number of sample points is $N = N_\theta N_\phi$. In applications, typically $N_\theta \sim N_\phi \sim \sqrt{N}$. Since the total number of spherical harmonics in the sum (6.7.12) is $l_{\max}^2$, we also have $l_{\max} \sim \sqrt{N}$.

How many operations does it take to evaluate the sum (6.7.12)? Direct evaluation of $l_{\max}^2$ terms at N sample points is an $O(N^2)$ process. You might try to speed this up by choosing the sample points ϕ_j to be equally spaced in angle and doing the sum over m by an FFT. Each FFT is $O(N_\phi \ln N_\phi)$, and you have to do $O(N_\theta l_{\max})$ of them, for a total of $O(N^{3/2} \ln N)$ operations, which is some improvement. A simple rearrangement [3-5] gives an even better way: Interchange the order of summation

$$\sum_{l=0}^{l_{\max}} \sum_{m=-l}^l \longleftrightarrow \sum_{m=-l_{\max}}^{l_{\max}} \sum_{l=|m|}^{l_{\max}} \quad (6.7.13)$$

so that

$$f(\theta_i, \phi_j) = \sum_{m=-l_{\max}}^{l_{\max}} q_m(\theta_i) e^{im\phi_j} \quad (6.7.14)$$

where

$$q_m(\theta_i) = \sum_{l=|m|}^{l_{\max}} a_{lm} \tilde{P}_l^m(\cos \theta_i) \quad (6.7.15)$$

Evaluating the sum in (6.7.15) is $O(l_{\max})$, and one must do this for $O(l_{\max} N_\theta)$ q_m 's, so the total work is $O(N^{3/2})$. To evaluate equation (6.7.14) by an FFT at fixed θ_i is $O(N_\phi \ln N_\phi)$. There are N_θ FFTs to be done, for a total operations count of $O(N \ln N)$, which is negligible in comparison. So the total algorithm is $O(N^{3/2})$. Note that you can evaluate equation (6.7.14) either by precomputing and storing the $\tilde{P}_l^m$'s using the recurrence relation (6.7.9), or by Clenshaw's method (§5.4).

What about inverting the transform (6.7.12)? The formal inverse for the expansion of a continuous function $f(\theta, \phi)$ follows from the orthonormality of the Y_{lm} 's, equation (6.7.1),

$$a_{lm} = \int \sin \theta \, d\theta \, d\phi \, f(\theta, \phi) e^{-im\phi} \tilde{P}_l^m(\cos \theta) \quad (6.7.16)$$

For the discrete case, where we have a sampled function, the integral becomes a quadrature:

$$a_{lm} = \sum_{i,j} w(\theta_i) f(\theta_i, \phi_j) e^{-im\phi_j} \tilde{P}_l^m(\cos \theta_i) \quad (6.7.17)$$

Here $w(\theta_i)$ are the quadrature weights. In principle we could consider weights that depend on ϕ_j as well, but in practice we do the ϕ quadrature by an FFT, so the weights are unity. A good choice for the weights for an equi-angular grid in θ is given in Ref. [3], Theorem 3. Another possibility is to use Gaussian quadrature for the θ integral. In this case, you choose the sample points so that the $\cos \theta_i$'s are the abscissas returned by `gauleg` and the $w(\theta_i)$'s are the corresponding weights. The best way to organize the calculation is to first do the FFTs, computing

$$g_m(\theta_i) = \sum_j f(\theta_i, \phi_j) e^{-im\phi_j} \quad (6.7.18)$$

Then

$$a_{lm} = \sum_i w(\theta_i) g_m(\theta_i) \tilde{P}_l^m(\cos \theta_i) \quad (6.7.19)$$